CACS355 · BCA VI Semester · Network Programming

# Unit 1: Introduction

Theory, exam programs and interview notes, in **Java** (for the exam) and **Go** (for jobs), side by side.

| Syllabus topic | Where it is in this guide |
|---|---|
| 1.1 Network Programming Features and Scope | Part A: A1, A2 |
| 1.2 Network Programming Language, Tools & Platforms | Part A: A3 (+ B1 Java, C1 Go) |
| 1.3 Client and Server Applications | Part A: A4 (+ programs in B and C) |
| 1.4 Client server model and software design | Part A: A5 (+ programs in B and C) |

**How to read this guide**

**Part A, Theory** is the same for both languages. Read it first. It is your written-answer material for Unit 1.

**Part B, Java** is the exam track: the three client/server programs you should be able to write on paper.

**Part C, Go** is the job track: the same three programs in Go, plus what interviewers ask about them.

**Part D** lists likely exam questions with answer pointers. **Part E** is a one-page Java vs Go cheat sheet.

Read B and C in parallel: Java program 1, then Go program 1, and so on. The programs are in the `java` and go folders next to this PDF, and all of them were compiled and run.

Unit 1 carries 3 teaching hours, so it is mostly short theory questions. But the client/server programs introduced here (TCP server, TCP client, UDP, multithreaded server) come back in Units 6, 7 and 10, which are among the heaviest units. Learning them now pays off three times.

# Contents

**Part A: Theory (common to Java and Go)** **3**

A1. What is network programming? . . . 3

A2. Features and scope of network programming (1.1) . . . 4

A3. Languages, tools and platforms (1.2) . . . 5

A4. Client and server applications (1.3) . . . 6

A5. Client-server model and software design (1.4) . . . 7

**Part B: Java (exam track)** **11**

B1. Why Java for network programming . . . 11

B2. The Java I/O pattern to memorise . . . 11

B3. Program 1: Iterative TCP echo server and client . . . 12

B4. Program 2: Multithreaded (concurrent) TCP server . . . 14

B5. Program 3: UDP server and client . . . 15

B6. Tips for writing Java code in the exam . . . 16

**Part C: Go (job track)** **17**

C1. Why Go for network programming . . . 17

C2. The Go I/O pattern to memorise . . . 17

C3. Program 1: Iterative TCP echo server and client . . . 18

C4. Program 2: Concurrent TCP server (goroutine per client) . . . 20

C5. Program 3: UDP server and client . . . 21

C6. Interview notes (Go) . . . 22

**Part D: Likely exam questions for Unit 1** **23**

Short questions . . . 23

Long questions . . . 23

**Part E: Java vs Go cheat sheet (Unit 1)** **24**

# Part A: Theory (common to Java and Go)

## A1. What is network programming?

**Network programming** is writing programs that communicate with other programs over a computer network. The two programs (called **processes**) may run on different machines across the Internet or on the same machine. They exchange data by following an agreed set of rules called a **protocol**, using an operating-system endpoint called a **socket**.

In one line for the exam: *network programming is the development of software that enables processes on different hosts to exchange data over a network using protocols such as TCP, UDP and HTTP, usually through the socket API.*

### Key terms you must know

| Term | Meaning | Example |
|---|---|---|
| Host | Any device on a network with an IP address. | Your laptop, a web server |
| IP address | Logical address that identifies a host. IPv4 is 32-bit, IPv6 is 128-bit. | 192.168.1.10, ::1 |
| Port | 16-bit number (0-65535) that identifies a process/service on a host. 0-1023 well-known, 1024-49151 registered, 49152-65535 dynamic/ephemeral. | 80 HTTP, 443 HTTPS, 22 SSH |
| Protocol | Rules for format and order of messages and the actions taken on them. | TCP, UDP, HTTP, FTP, DNS |
| Socket | One endpoint of a two-way communication link = IP address + port + protocol. | 127.0.0.1:5000 (TCP) |
| Socket pair | The 4-tuple that uniquely identifies a TCP connection: (client IP, client port, server IP, server port). | (10.0.0.5, 51234, 93.184.216.34, 443) |

### Where network programs sit: the TCP/IP model

| Layer | Job | Examples | Who handles it |
|---|---|---|---|
| Application | Meaning of the data | HTTP, FTP, SMTP, DNS, your own protocol | **You, the programmer** |
| Transport | Process-to-process delivery, ports | TCP, UDP | OS kernel, used via the socket API |
| Internet (Network) | Host-to-host delivery, routing | IP, ICMP | OS kernel and routers |
| Network Access (Link) | Frames on the physical medium | Ethernet, Wi-Fi | NIC driver and hardware |

A network programmer works at the **application layer** and talks to the transport layer through the **socket API**. You never build IP packets by hand. You open a socket, write bytes, and the OS does the rest.

### TCP vs UDP (asked in almost every network exam)

| Basis | TCP (Transmission Control Protocol) | UDP (User Datagram Protocol) |
|---|---|---|
| Connection | Connection-oriented (3-way handshake first) | Connectionless (just send) |
| Reliability | Reliable: ACKs, retransmission | Unreliable: packets may be lost |
| Ordering | Data arrives in order | No ordering guarantee |
| Data unit | Byte stream (no message boundaries) | Datagram (each message is separate) |

| Basis | TCP (Transmission Control Protocol) | UDP (User Datagram Protocol) |
|---|---|---|
| Flow/congestion control | Yes | No |
| Speed / overhead | Slower, 20-byte header | Faster, 8-byte header |
| Used by | HTTP, HTTPS, FTP, SMTP, SSH | DNS, DHCP, VoIP, video streaming, online games |
| Java classes | Socket, ServerSocket | DatagramSocket, DatagramPacket |
| Go functions | net.Dial("tcp"), net.Listen("tcp") | net.Dial("udp"), net.ListenPacket("udp") |

## A2. Features and scope of network programming (1.1)

### Features

- **Inter-process communication across machines.** Lets processes on different hosts exchange data as if they were local.
- **Protocol-based.** Communication follows standard protocols (TCP, UDP, HTTP, FTP, SMTP), so programs written in different languages on different OSes can still talk. A Go client can talk to a Java server.
- **Client-server and peer-to-peer support.** Programs can act as clients, servers, or both.
- **Socket abstraction.** A network connection is treated like a file or stream: open, read, write, close.
- **Concurrency.** Servers handle many clients at once using threads, processes, goroutines or event loops.
- **Platform independence.** High-level languages (Java, Go, Python) hide OS differences such as Winsock vs BSD sockets.
- **Resource sharing.** Files, printers, databases and computing power can be shared over the network.
- **Security support.** Encryption (SSL/TLS), authentication and access control can be built in (Unit 8).
- **Error handling and timeouts.** The network is unreliable, so programs must handle failures, delays and lost connections.
- **Scalability.** Well-designed network programs can grow from one user to millions (load balancing, distributed servers).

### Scope (where network programming is used)

| Area | Examples |
|---|---|
| Web applications | Web browsers, web servers, REST APIs, web crawlers |
| Communication | Email (SMTP/IMAP), chat apps, video calling, VoIP |
| File transfer and sharing | FTP, cloud storage sync, BitTorrent (P2P) |
| Distributed systems and cloud | Microservices, RPC/RMI/gRPC, distributed databases, Kubernetes |
| Remote access and administration | SSH, Telnet, remote desktop, network monitoring tools |
| Online games and streaming | Multiplayer game servers (often UDP), live video streaming |
| IoT and embedded | Smart devices sending sensor data to servers (MQTT, CoAP) |
| E-commerce and banking | Online payment, ATM networks, stock trading systems |
| Network security | Firewalls, VPNs, intrusion detection, TLS servers |

## A3. Languages, tools and platforms (1.2)

### Languages

| Language | Networking support | Typical use |
|---|---|---|
| C | BSD sockets API directly (socket, bind, listen, accept). Full control, more code. | OS kernels, routers, high-performance servers |
| C++ | BSD sockets, Boost.Asio | Game servers, trading systems |
| **Java** | java.net, java.nio, java.rmi, javax.net.ssl. Platform-independent, built-in threads. | Enterprise servers, Android apps, **this course** |
| **Go** | net, net/http, crypto/tls in the standard library. Goroutines make concurrent servers easy. | Cloud services, microservices, Docker, Kubernetes |
| Python | socket, asyncio, requests. Very short code. | Scripting, automation, prototypes |
| C# | .NET System.Net, System.Net.Sockets | Windows and enterprise apps |
| JavaScript (Node.js) | net, http, WebSockets. Event-driven, non-blocking. | Real-time web apps, APIs |
| Rust | std::net, tokio. Memory-safe and fast. | Performance-critical network services |

### Tools

| Tool | Purpose |
|---|---|
| ping | Test whether a host is reachable (ICMP echo). |
| ipconfig (Windows) / ifconfig, ip (Linux) | Show IP address, network interfaces, gateway. |
| nslookup / dig | DNS lookup: hostname to IP address. |
| netstat / ss | Show open ports, active connections, listening sockets. |
| tracert / traceroute | Show the path (routers) packets take to a host. |
| telnet / netcat (nc) | Open a raw TCP connection to test a server manually (used in Unit 6). |
| curl / Postman | Send HTTP requests and inspect responses. |
| Wireshark / tcpdump | Capture and analyse packets on the network. |
| IDE and SDK | JDK + IntelliJ / Eclipse / NetBeans / VS Code for Java; Go toolchain + VS Code / GoLand for Go. |

### Platforms

- **Unix/Linux:** BSD (POSIX) sockets. Most servers on the Internet run here.
- **Windows:** Winsock API, which is based on BSD sockets.
- **macOS, Android, iOS:** BSD-style sockets underneath; Android apps commonly use Java/Kotlin networking.
- **Virtual machines and runtimes:** the JVM lets the same Java bytecode run on any OS. Go compiles to a native binary for each OS.
- **Cloud and containers:** AWS, Azure, GCP, Docker, Kubernetes. Network programs are deployed as services here.

## A4. Client and server applications (1.3)

A **client** is a program that **initiates** communication and requests a service. A **server** is a program that **waits** for requests and provides the service. The words describe a *role* in one conversation, not a type of computer: the same machine can run both.

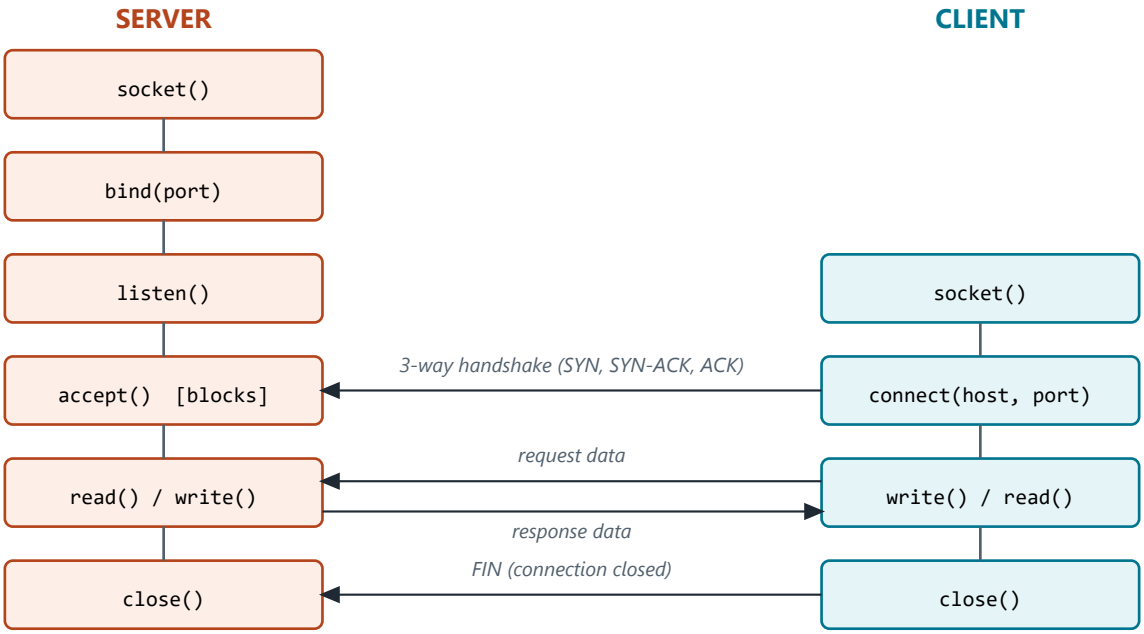
| Basis | Client | Server |
|---|---|---|
| Role | Active: starts the communication | Passive: waits for clients |
| Port | Usually a temporary (ephemeral) port chosen by the OS | A fixed, well-known port, e.g. 80, 443, 5000 |
| Lifetime | Runs when the user needs it, then exits | Runs continuously (often as a daemon/service) |
| Number | Talks to one server at a time (usually) | Serves many clients, often at the same time |
| Privilege | Ordinary user privileges | Often needs special privileges (ports below 1024, files, DB) |
| Complexity | Simpler | More complex: concurrency, security, logging, reliability |
| Examples | Web browser, email client, FTP client, ssh | Apache/Nginx, mail server, FTP server, sshd |

### Common client/server applications and their ports

| Service | Client | Server | Protocol / Port |
|---|---|---|---|
| Web | Browser, curl | Apache, Nginx | HTTP 80, HTTPS 443 (TCP) |
| Email | Outlook, Thunderbird | Mail server | SMTP 25, POP3 110, IMAP 143 (TCP) |
| File transfer | FileZilla | FTP server | FTP 20/21 (TCP) |
| Remote login | ssh, PuTTY | sshd | SSH 22, Telnet 23 (TCP) |
| Name lookup | Resolver in the OS | DNS server | DNS 53 (mostly UDP) |
| Address assignment | DHCP client | DHCP server | DHCP 67/68 (UDP) |
| Database | App / DB client | MySQL, PostgreSQL | 3306, 5432 (TCP) |

### Steps of a TCP client and server

These six server steps and four client steps are the skeleton of every TCP program in this course. Java and Go hide some of them in one call, as shown in the table after the diagram.

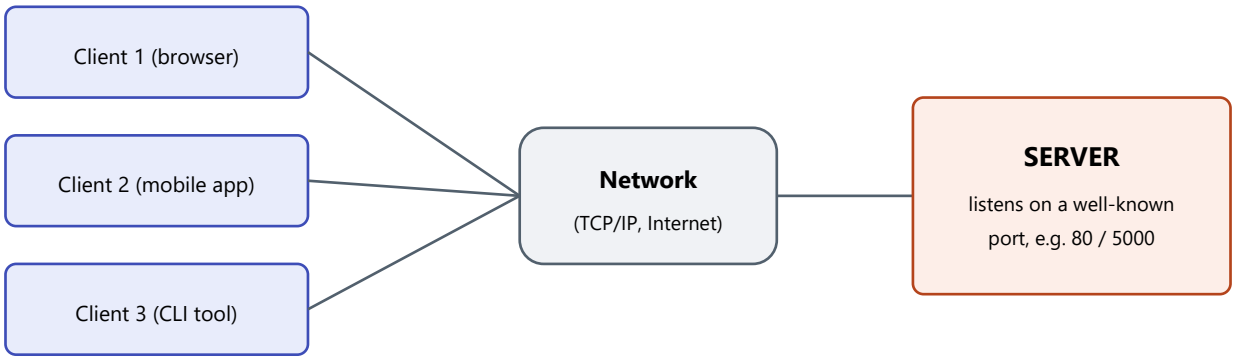



| Step | C / BSD sockets | Java | Go |
|---|---|---|---|
| Create + bind + listen | socket(), bind(), listen() | new ServerSocket(port) | net.Listen("tcp", ":port") |
| Wait for a client | accept() | server.accept() | ln.Accept() |
| Client connects | socket(), connect() | new Socket(host, port) | net.Dial("tcp", "host:port") |
| Exchange data | read(), write() | getInputStream(), getOutputStream() | conn.Read(), conn.Write() (net.Conn) |
| Close | close() | socket.close() | conn.Close() |

**UDP flow** is simpler: no listen, accept or connect. The server creates a socket bound to a port and loops on *receive -> process -> send reply to the sender's address*. The client creates a socket and does *send -> receive*. Each message is an independent datagram.

## A5. Client-server model and software design (1.4)

The **client-server model** is a distributed application structure in which tasks are divided between **servers**, which provide a resource or service, and **clients**, which request it. Clients and servers communicate over a network using a request-response pattern.




### Characteristics

- **Request-response:** the client sends a request; the server processes it and sends a response.
- **Asymmetric roles:** the server is centralised and waits; clients are many and initiate.
- **Centralised resources:** data and logic live on the server, so they are easy to manage, back up and secure.
- **Transparency:** the client does not need to know how the server works internally, only the protocol.
- **Scalability:** more clients can be added; the server can be scaled up or replicated.

### Advantages and disadvantages

| Advantages | Disadvantages |
|---|---|
| Centralised control of data, security and backups | Server is a single point of failure |
| Easy to update the service in one place | Server can become a bottleneck under heavy load (congestion) |
| Clients can be thin and simple | Server hardware and maintenance are expensive |
| Better security: access control on the server | Needs a network: no connection means no service |
| Scales by adding server capacity | Needs skilled administrators |

### Client-server vs peer-to-peer

| Basis | Client-server | Peer-to-peer (P2P) |
|---|---|---|
| Roles | Fixed: dedicated server, many clients | Every peer is both client and server |
| Data | Centralised on the server | Distributed across peers |
| Failure | Server down means service down | No single point of failure |
| Security and control | Easier | Harder |
| Cost | Expensive server | Cheap, uses peers' resources |
| Examples | Web, email, banking | BitTorrent, blockchain networks |

### Tiered architectures

- **2-tier:** client talks directly to the server/database. Example: a desktop app connected to a MySQL server.
- **3-tier:** presentation tier (client UI) -> application/logic tier (app server) -> data tier (database). Most web apps.
- **N-tier:** more layers such as load balancers, caches, microservices and message queues. Large systems like Netflix.

### Software design issues for client-server programs

These come from Comer and Stevens (reference book 2). They are the most important exam material in Unit 1, and the programs in Parts B and C show each one in code.

### 1. Connection-oriented vs connectionless

| Connection-oriented (TCP) | Connectionless (UDP) |
|---|---|
| A connection is set up before data is sent and closed afterwards | Each message is sent independently, no setup |
| Reliable, ordered: the protocol handles lost data | Application must handle loss, duplication and ordering |
| Server keeps a socket per client | One socket serves all clients |

| Connection-oriented (TCP) | Connectionless (UDP) |
|---|---|
| Use when correctness matters: file transfer, web, email | Use when speed matters or messages are tiny: DNS, streaming, games |

### 2. Iterative vs concurrent servers

An **iterative server** handles **one client at a time**: it accepts a client, serves it fully, then accepts the next. A **concurrent server** handles **many clients at the same time**, usually by creating a new thread, process or goroutine for each client.

| Iterative server | Concurrent server |
|---|---|
| One request at a time | Many requests at the same time |
| Simple to write and debug | More complex: threads, shared data, synchronisation |
| Other clients wait in a queue; a slow client blocks everyone | A slow client only blocks its own thread |
| Good for short requests (e.g. daytime, DNS-style UDP) | Needed for long sessions (web, chat, file transfer) |
| Java: serve inside the accept loop | Java: new Thread(handler).start() per client |
| Go: handle(conn) | Go: go handle(conn) |

### 3. The four basic server types

Combining issues 1 and 2 gives four server designs. This 2x2 table is a classic exam answer.

| | Connectionless (UDP) | Connection-oriented (TCP) |
|---|---|---|
| **Iterative** | **Iterative connectionless**<br>Most common UDP design. Each request is short. Example: UDP echo server (Program 3). | **Iterative connection-oriented**<br>Simple, but one client blocks others. Example: TCP echo server (Program 1). |
| **Concurrent** | **Concurrent connectionless**<br>Rare. Only used when each request needs long processing. | **Concurrent connection-oriented**<br>Most common TCP design. Example: multithreaded server (Program 2), web servers. |

### 4. Stateful vs stateless servers

| Stateful server | Stateless server |
|---|---|
| Remembers information about each client between requests (session, login, file position) | Treats every request independently; keeps no client history |
| Requests can be shorter (server remembers context) | Each request must carry all needed information |
| Crash loses state; harder to scale and recover | Easy to scale and recover: any server can answer |
| Examples: FTP (current directory), SSH, online game sessions | Examples: HTTP (basic), DNS, NFS v3 |

### 5. Server complexity and privilege

Servers usually run with special privileges, so they must handle:

- **Authentication:** verify who the client is.
- **Authorization:** check that the client is allowed to access the resource.
- **Data security:** make sure data is not unintentionally revealed or changed.
- **Privacy:** keep information about individuals from unauthorized access.

- **Protection:** make sure clients cannot abuse system resources (e.g. too many connections, huge requests).

### 6. Client design issues

- **Locating the server:** hard-coded in the program, given as a command-line argument, read from a config file, or found via DNS / a directory service.
- **Parameterization:** a good client takes the host and port as parameters instead of hard-coding them, so it can be reused (e.g. telnet host port).
- **Standard vs non-standard clients:** standard clients use well-known services (HTTP, SMTP); non-standard clients use a locally defined protocol.
- **Choosing a local port:** clients normally let the OS pick a free ephemeral port automatically.
- **Handling failure:** timeouts, retries, and clear error messages when the server is unreachable.

### 7. Ways to achieve concurrency in a server

| Approach | How | Pros / Cons |
|---|---|---|
| Process per client | fork() a new process for each client (classic Unix) | Strong isolation; heavy and slow to create |
| Thread per client | New thread for each client (Java Program 2) | Simple; thousands of threads use lots of memory |
| Thread pool | Fixed set of worker threads (Java ExecutorService) | Controls resource use; clients may wait for a free thread |
| Event-driven / non-blocking I/O | One thread watches many sockets with select/epoll (Java NIO Selector, Unit 9) | Very scalable; harder to code |
| Goroutine per client | go handle(conn): Go runtime multiplexes goroutines onto threads and uses epoll internally | Simple code and very scalable |

# Part B: Java (exam track)

## B1. Why Java for network programming

- **Platform independent:** the same bytecode runs on Windows, Linux and macOS through the JVM ("write once, run anywhere").
- **Rich built-in networking library:** the java.net package hides low-level socket details.
- **Built-in multithreading:** the Thread class and ExecutorService make concurrent servers straightforward.
- **Security:** SSL/TLS support (javax.net.ssl), a security manager and a strongly typed language.
- **Exception handling:** network errors are reported as checked exceptions (IOException), so the compiler forces you to handle them.
- **Object-oriented and robust:** automatic garbage collection, no pointer bugs.
- **Distributed computing:** RMI lets a program call methods on objects in another JVM (Unit 12).

### Java networking packages and classes (map to later units)

| Class / package | Purpose | Unit |
|---|---|---|
| InetAddress, Inet4Address, Inet6Address | Represent IP addresses, DNS lookup | 2 |
| NetworkInterface | List local network interfaces | 2 |
| URL, URI, URLEncoder, URLDecoder | Work with web addresses | 3 |
| CookieManager, CookieStore | HTTP cookies | 4 |
| URLConnection, HttpURLConnection | Talk to web servers | 5 |
| Socket | TCP client | 6 |
| ServerSocket | TCP server | 7 |
| javax.net.ssl: SSLSocket, SSLServerSocket | Secure (TLS) sockets | 8 |
| java.nio: ByteBuffer, SocketChannel, Selector | Non-blocking I/O | 9 |
| DatagramSocket, DatagramPacket, DatagramChannel | UDP | 10 |
| MulticastSocket | IP multicast | 11 |
| java.rmi: Remote, UnicastRemoteObject, Registry | Remote Method Invocation | 12 |

## B2. The Java I/O pattern to memorise

Almost every Java socket program in the exam uses these same lines. Memorise them once and you save time on every program.

```
import java.io.*;
import java.net.*;

// read text lines from the socket
BufferedReader in = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));

// write text lines to the socket (true = auto-flush after println)
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

String line = in.readLine();   // returns null when the other side closes
out.println("reply");
```

- **Why wrap the streams?** A socket gives raw bytes (InputStream). InputStreamReader turns bytes into characters, and BufferedReader adds readLine().
- **Why auto-flush (true)?** Without it, println() stays in a buffer and the other side waits forever. This is the most common bug in exam programs.
- **Exceptions to know:** IOException (general I/O), UnknownHostException (bad hostname), ConnectException (no server listening), BindException (port already in use), SocketTimeoutException (timeout expired).

## B3. Program 1: Iterative TCP echo server and client

**Design:** iterative, connection-oriented. The server echoes back each line the client sends and handles one client at a time. Typical exam wording: "*Write a client-server program in Java in which the client sends a message and the server replies.*"

`EchoServer.java`

```
import java.io.*;
import java.net.*;

// Iterative, connection-oriented (TCP) server: handles ONE client at a time.
public class EchoServer {
    public static void main(String[] args) {
        int port = 5000;
        try (ServerSocket server = new ServerSocket(port)) {
            System.out.println("Server started on port " + port);
            while (true) {
                Socket client = server.accept();          // blocks until a client connects
                System.out.println("Client connected: " + client.getInetAddress());

                BufferedReader in = new BufferedReader(
                        new InputStreamReader(client.getInputStream()));
                PrintWriter out = new PrintWriter(client.getOutputStream(), true);

                String line;
                while ((line = in.readLine()) != null) {  // null = client closed connection
                    System.out.println("Received: " + line);
                    out.println("Echo: " + line);
                    if (line.equalsIgnoreCase("bye")) break;
                }
                client.close();
                System.out.println("Client disconnected");
            }
        } catch (IOException e) {
            System.out.println("Server error: " + e.getMessage());
        }
    }
}
```

`EchoClient.java`

```
import java.io.*;
import java.net.*;

// TCP client: sends lines typed by the user, prints the server's reply.
public class EchoClient {
    public static void main(String[] args) {
        String host = "localhost";
        int port = 5000;
        try (Socket socket = new Socket(host, port)) {
            System.out.println("Connected to " + host + ":" + port);

            BufferedReader in = new BufferedReader(
                    new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
```

```
            BufferedReader keyboard = new BufferedReader(
                    new InputStreamReader(System.in));

            String msg;
            while ((msg = keyboard.readLine()) != null) {
                out.println(msg);                          // send to server
                System.out.println("Server: " + in.readLine());  // wait for reply
                if (msg.equalsIgnoreCase("bye")) break;
            }
        } catch (UnknownHostException e) {
            System.out.println("Unknown host: " + host);
        } catch (IOException e) {
            System.out.println("I/O error: " + e.getMessage());
        }
    }
}
```

### How it works

- `new ServerSocket(5000)` creates the socket, binds it to port 5000 and starts listening (C's socket + bind + listen in one call).
- `server.accept()` blocks until a client connects, then returns a new `Socket` just for that client.
- `new Socket("localhost", 5000)` on the client performs the TCP 3-way handshake.
- Both sides wrap the socket streams with BufferedReader/PrintWriter and exchange lines.
- `readLine()` returning `null` means the other side closed the connection.
- **Why iterative?** The inner while loop must finish before accept() runs again, so a second client waits.
- `try (...)` (try-with-resources) closes the socket automatically, even when an exception is thrown.

### How to run

```
javac EchoServer.java EchoClient.java
java EchoServer          (terminal 1)
java EchoClient          (terminal 2, type lines, "bye" to quit)
```

Sample output

```
Connected to localhost:5000
hello
Server: Echo: hello
world
Server: Echo: world
bye
Server: Echo: bye
```

## B4. Program 2: Multithreaded (concurrent) TCP server

**Design:** concurrent, connection-oriented. Each client gets its own thread, so many clients are served at once. Typical exam wording: "*Write a multithreaded server that can handle multiple clients simultaneously.*" The client from Program 1 works with this server; just change the port to 5001.

`MultiThreadedServer.java`

```
import java.io.*;
import java.net.*;

// Concurrent, connection-oriented (TCP) server: one thread per client.
public class MultiThreadedServer {
    public static void main(String[] args) {
        int port = 5001;
        try (ServerSocket server = new ServerSocket(port)) {
            System.out.println("Multithreaded server started on port " + port);
            while (true) {
                Socket client = server.accept();
                System.out.println("New client: " + client.getInetAddress());
                new Thread(new ClientHandler(client)).start();  // serve in a new thread
            }
        } catch (IOException e) {
            System.out.println("Server error: " + e.getMessage());
        }
    }
}

class ClientHandler implements Runnable {
    private Socket client;

    ClientHandler(Socket client) {
        this.client = client;
    }

    public void run() {
        try {
            BufferedReader in = new BufferedReader(
                    new InputStreamReader(client.getInputStream()));
            PrintWriter out = new PrintWriter(client.getOutputStream(), true);

            String line;
            while ((line = in.readLine()) != null) {
                System.out.println(Thread.currentThread().getName() + " got: " + line);
                out.println("Echo: " + line);
                if (line.equalsIgnoreCase("bye")) break;
            }
            client.close();
        } catch (IOException e) {
            System.out.println("Client error: " + e.getMessage());
        }
    }
}
```

### How it works

- The main thread only does one job: `accept()` clients in a loop.
- For each client it creates a `ClientHandler` (which `implements Runnable`) and starts a new Thread.
- The `run()` method contains the same echo loop as Program 1, but now it runs in parallel for every client.
- **Difference from Program 1 in one line:** the echo loop moved from main() into run(), and main() calls `new Thread(...).start()`.
- You could also write `class ClientHandler extends Thread` and call `new ClientHandler(client).start()`. Both are accepted in exams.

### Better version: thread pool (good for extra marks)

Creating unlimited threads can exhaust memory. A thread pool reuses a fixed number of threads:

```
import java.util.concurrent.*;

ExecutorService pool = Executors.newFixedThreadPool(10);   // at most 10 worker threads
while (true) {
    Socket client = server.accept();
    pool.execute(new ClientHandler(client));               // instead of new Thread(...).start()
}
```

Sample output

```
Multithreaded server started on port 5001
New client: /127.0.0.1
New client: /127.0.0.1
New client: /127.0.0.1
Thread-0 got: client1
Thread-2 got: client0
Thread-1 got: client2
```

Three clients connected at the same time and were served by three different threads.

## B5. Program 3: UDP server and client

**Design:** iterative, connectionless. No connection is made; the server receives a datagram, reads the sender's address from it, and sends a reply to that address. UDP is covered in depth in Unit 10, but "compare TCP and UDP with a program" can appear here.

**UDPServer.java**

```
import java.net.*;

// Iterative, connectionless (UDP) server: replies to each datagram.
public class UDPServer {
    public static void main(String[] args) {
        int port = 5002;
        try (DatagramSocket socket = new DatagramSocket(port)) {
            System.out.println("UDP server started on port " + port);
            byte[] buffer = new byte[1024];
            while (true) {
                DatagramPacket request = new DatagramPacket(buffer, buffer.length);
                socket.receive(request);                  // blocks until a datagram arrives
                String msg = new String(request.getData(), 0, request.getLength());
                System.out.println("Received: " + msg);

                byte[] reply = ("Echo: " + msg).getBytes();
                DatagramPacket response = new DatagramPacket(reply, reply.length,
                        request.getAddress(), request.getPort());  // send back to sender
                socket.send(response);
            }
        } catch (Exception e) {
            System.out.println("Server error: " + e.getMessage());
        }
    }
}
```

**UDPClient.java**

```
import java.net.*;

// UDP client: sends one datagram and waits for the reply.
public class UDPClient {
    public static void main(String[] args) {
        try (DatagramSocket socket = new DatagramSocket()) {
            InetAddress server = InetAddress.getByName("localhost");
```

```
            int port = 5002;

            byte[] data = "Hello UDP".getBytes();
            DatagramPacket request = new DatagramPacket(data, data.length, server, port);
            socket.send(request);

            byte[] buffer = new byte[1024];
            DatagramPacket response = new DatagramPacket(buffer, buffer.length);
            socket.setSoTimeout(3000);                    // UDP may lose packets: don't wait forever
            socket.receive(response);
            System.out.println("Server: " + new String(response.getData(), 0, response.getLength()));
        } catch (SocketTimeoutException e) {
            System.out.println("No reply (packet lost?)");
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

### How it works

- `DatagramSocket(5002)` binds the server to port 5002. There is no accept(): one socket serves every client.
- `DatagramPacket` is the "envelope": data + length + (for sending) destination address and port.
- `receive()` fills the packet and also tells you who sent it: `getAddress()` and `getPort()`.
- Use `new String(data, 0, getLength())`, not the whole buffer, otherwise you get leftover bytes.
- `setSoTimeout(3000)` on the client: UDP packets can be lost, so never wait forever.

Sample output

```
Server: Echo: Hello UDP
```

## B6. Tips for writing Java code in the exam

- Always write `import java.io.*;` and `import java.net.*;` at the top.
- Wrap network code in `try { ... } catch (IOException e) { ... }`. Missing exception handling loses marks.
- Write the server first, then the client. Use the same port number in both and mention it.
- Add short comments next to the key calls (accept, getInputStream, println). Examiners look for them.
- Write a small "Output" box after the program, as shown above.
- For a multithreaded server, clearly show `implements Runnable` / `extends Thread` and `start()`.
- If you forget the exact code, write the TCP steps (Part A4) as comments first and fill in the Java calls. This earns partial marks.

# Part C: Go (job track)

Same three programs, same ports, same behaviour. A Go client can talk to the Java server and the other way round, which shows that the **protocol**, not the language, defines the communication.

## C1. Why Go for network programming

- **Networking in the standard library:** net, net/http, net/url, crypto/tls, net/rpc. No external packages needed for this whole syllabus.
- **Goroutines:** lightweight threads (start at about 2 KB of stack vs about 1 MB for an OS thread). A server can run hundreds of thousands of them.
- **Blocking-style code, non-blocking underneath:** you write simple Read/Write calls; the Go runtime's network poller uses epoll/kqueue/IOCP internally.
- **Single static binary:** easy to deploy in Docker and Kubernetes. Docker, Kubernetes, etcd, Prometheus and Caddy are written in Go.
- **Explicit error handling:** every network call returns an error value you must check.
- **Cross-platform:** the same code compiles for Windows, Linux and macOS (GOOS/GOARCH).

### Go packages mapped to the syllabus

| Java | Go equivalent | Unit |
|---|---|---|
| InetAddress, NetworkInterface | net.IP, net.LookupIP, net.Interfaces | 2 |
| URL, URI, URLEncoder | net/url: url.Parse, ResolveReference, QueryEscape | 3 |
| CookieManager | net/http/cookiejar | 4 |
| URLConnection, HttpURLConnection | net/http: http.Client, http.NewRequest | 5 |
| Socket | net.Dial("tcp", ...) returning net.Conn | 6 |
| ServerSocket | net.Listen("tcp", ...) returning net.Listener | 7 |
| SSLSocket, SSLServerSocket | crypto/tls: tls.Dial, tls.Listen | 8 |
| java.nio Selector | Not needed: goroutines + runtime netpoller | 9 |
| DatagramSocket | net.ListenPacket("udp", ...), net.Dial("udp", ...) | 10 |
| MulticastSocket | net.ListenMulticastUDP, golang.org/x/net/ipv4 | 11 |
| RMI | net/rpc (stdlib); gRPC in industry | 12 |

## C2. The Go I/O pattern to memorise

```
conn, err := net.Dial("tcp", "localhost:5000")   // or ln.Accept() on the server
if err != nil {
    log.Fatal(err)
}
defer conn.Close()                    // runs when the function returns

scanner := bufio.NewScanner(conn)     // read lines (like BufferedReader)
for scanner.Scan() {
    line := scanner.Text()
    fmt.Fprintln(conn, "reply")       // write a line (like PrintWriter.println)
}
```

- **net.Conn** is an interface with Read, Write and Close. It is both an io.Reader and an io.Writer, so every Go I/O helper works on it directly. Java needs stream wrappers; Go doesn't.

- **No auto-flush problem:** conn.Write sends immediately. (If you use bufio.Writer you must call Flush, which is the same idea as Java's auto-flush.)
- **Errors are values:** `if err != nil` replaces try/catch. io.EOF means the other side closed, like Java's readLine() returning null.
- **defer** replaces try-with-resources/finally for closing sockets.

## C3. Program 1: Iterative TCP echo server and client

**go/echo_server/main.go**

```
package main

import (
    "bufio"
    "fmt"
    "log"
    "net"
    "strings"
)

// Iterative, connection-oriented (TCP) server: handles ONE client at a time.
func main() {
    ln, err := net.Listen("tcp", ":5000")
    if err != nil {
        log.Fatal(err)
    }
    defer ln.Close()
    fmt.Println("Server started on port 5000")

    for {
        conn, err := ln.Accept() // blocks until a client connects
        if err != nil {
            log.Println("accept:", err)
            continue
        }
        handle(conn) // no "go" keyword -> iterative
    }
}

func handle(conn net.Conn) {
    defer conn.Close()
    fmt.Println("Client connected:", conn.RemoteAddr())

    scanner := bufio.NewScanner(conn)
    for scanner.Scan() { // false when client closes connection
        line := scanner.Text()
        fmt.Println("Received:", line)
        fmt.Fprintln(conn, "Echo: "+line)
        if strings.EqualFold(line, "bye") {
            break
        }
    }
    fmt.Println("Client disconnected")
}
```

**go/echo_client/main.go**

```
package main

import (
    "bufio"
    "fmt"
    "log"
    "net"
    "os"
    "strings"
```

```
)

// TCP client: sends lines typed by the user, prints the server's reply.
func main() {
    conn, err := net.Dial("tcp", "localhost:5000")
    if err != nil {
        log.Fatal(err)
    }
    defer conn.Close()
    fmt.Println("Connected to", conn.RemoteAddr())

    server := bufio.NewReader(conn)
    keyboard := bufio.NewScanner(os.Stdin)

    for keyboard.Scan() {
        msg := keyboard.Text()
        fmt.Fprintln(conn, msg) // send to server

        reply, err := server.ReadString('\n') // wait for reply
        if err != nil {
            log.Fatal(err)
        }
        fmt.Print("Server: ", reply)
        if strings.EqualFold(msg, "bye") {
            break
        }
    }
}
```

### How it works / compared with Java

| Java | Go |
|---|---|
| new ServerSocket(5000) | net.Listen("tcp", ":5000") |
| server.accept() returns Socket | ln.Accept() returns net.Conn, error |
| new Socket("localhost", 5000) | net.Dial("tcp", "localhost:5000") |
| BufferedReader.readLine() | bufio.Scanner.Scan() + Text() |
| PrintWriter.println(s) | fmt.Fprintln(conn, s) |
| readLine() == null | Scan() returns false (EOF) |
| try-with-resources | defer conn.Close() |
| catch (IOException e) | if err != nil { ... } |

- The server is iterative because main() calls `handle(conn)` directly: the loop cannot accept the next client until handle returns.

### How to run

```
cd go
go run ./echo_server      (terminal 1)
go run ./echo_client      (terminal 2, type lines, "bye" to quit)
```

Sample output

```
Connected to [::1]:5000
hello
Server: Echo: hello
world
Server: Echo: world
bye
Server: Echo: bye
```

## C4. Program 2: Concurrent TCP server (goroutine per client)

go/concurrent_server/main.go

```
package main

import (
    "bufio"
    "fmt"
    "log"
    "net"
    "strings"
)

// Concurrent, connection-oriented (TCP) server: one goroutine per client.
func main() {
    ln, err := net.Listen("tcp", ":5001")
    if err != nil {
        log.Fatal(err)
    }
    defer ln.Close()
    fmt.Println("Concurrent server started on port 5001")

    for {
        conn, err := ln.Accept()
        if err != nil {
            log.Println("accept:", err)
            continue
        }
        fmt.Println("New client:", conn.RemoteAddr())
        go handle(conn) // "go" keyword -> serve in a new goroutine
    }
}

func handle(conn net.Conn) {
    defer conn.Close()
    scanner := bufio.NewScanner(conn)
    for scanner.Scan() {
        line := scanner.Text()
        fmt.Printf("%s got: %s\n", conn.RemoteAddr(), line)
        fmt.Fprintln(conn, "Echo: "+line)
        if strings.EqualFold(line, "bye") {
            return
        }
    }
}
```

### How it works / compared with Java

- The only change from the iterative server is **one word**: `go handle(conn)` instead of `handle(conn)`. Java needs a Runnable class, a Thread object and start().
- Each goroutine owns one connection. When handle returns, `defer conn.Close()` closes it.
- No thread pool is needed for most servers because goroutines are cheap. To limit concurrency, use a buffered channel as a semaphore (see the interview notes in C6).

Sample output

```
Concurrent server started on port 5001
New client: [::1]:56557
[::1]:56557 got: client1
New client: [::1]:56558
[::1]:56558 got: client2
New client: [::1]:56556
[::1]:56556 got: client0
```

## C5. Program 3: UDP server and client

**go/udp_server/main.go**

```
package main

import (
    "fmt"
    "log"
    "net"
)

// Iterative, connectionless (UDP) server: replies to each datagram.
func main() {
    conn, err := net.ListenPacket("udp", ":5002")
    if err != nil {
        log.Fatal(err)
    }
    defer conn.Close()
    fmt.Println("UDP server started on port 5002")

    buf := make([]byte, 1024)
    for {
        n, addr, err := conn.ReadFrom(buf) // blocks until a datagram arrives
        if err != nil {
            log.Println("read:", err)
            continue
        }
        msg := string(buf[:n])
        fmt.Println("Received:", msg)
        conn.WriteTo([]byte("Echo: "+msg), addr) // send back to sender
    }
}
```

**go/udp_client/main.go**

```
package main

import (
    "fmt"
    "log"
    "net"
    "time"
)

// UDP client: sends one datagram and waits for the reply.
func main() {
    conn, err := net.Dial("udp", "localhost:5002") // no handshake: just sets the default address
    if err != nil {
        log.Fatal(err)
    }
    defer conn.Close()

    conn.Write([]byte("Hello UDP"))

    conn.SetReadDeadline(time.Now().Add(3 * time.Second)) // UDP may lose packets: don't wait forever
    buf := make([]byte, 1024)
    n, err := conn.Read(buf)
    if err != nil {
        fmt.Println("No reply (packet lost?):", err)
        return
    }
    fmt.Println("Server:", string(buf[:n]))
}
```

### How it works / compared with Java

| Java | Go |
|---|---|
| new DatagramSocket(5002) | net.ListenPacket("udp", ":5002") |
| DatagramPacket + socket.receive(p) | n, addr, err := conn.ReadFrom(buf) |
| p.getAddress(), p.getPort() | addr (a net.Addr) |
| new String(p.getData(), 0, p.getLength()) | string(buf[:n]) |
| socket.send(new DatagramPacket(..., addr, port)) | conn.WriteTo(data, addr) |
| setSoTimeout(3000) | SetReadDeadline(time.Now().Add(3 * time.Second)) |

- On the client, `net.Dial("udp", ...)` does **not** perform a handshake. It only records the default destination, so Write and Read can be used. This is a common interview trick question.
- Go has no DatagramPacket object: a byte slice and an address do the same job.

Sample output

```
Server: Echo: Hello UDP
```

## C6. Interview notes (Go)

Unit 1 concepts that Go backend interviews ask about, with short answers:

**Q. What happens when you call net.Dial("tcp", ...)?**
DNS lookup if needed, then the OS performs the TCP 3-way handshake (SYN, SYN-ACK, ACK) from an ephemeral local port. Dial returns a net.Conn once the connection is established.

**Q. Goroutine vs OS thread?**
Goroutines are managed by the Go runtime, start with about 2 KB of stack that grows as needed, and are multiplexed onto a few OS threads (M:N scheduling). Thread creation and context switches are far more expensive.

**Q. If conn.Read blocks, does it block an OS thread?**
No. The runtime parks the goroutine and registers the socket with the netpoller (epoll on Linux). When data arrives the goroutine is woken. This is why goroutine-per-connection scales.

**Q. How do you limit the number of concurrent clients?**
Use a buffered channel as a semaphore: `sem := make(chan struct{}, 100)`; send to it before go handle and receive from it when handle finishes. Or use golang.org/x/sync/semaphore.

**Q. How do you stop a slow client from holding a connection forever?**
Set deadlines: `conn.SetReadDeadline(time.Now().Add(30*time.Second))`. Java's equivalent is setSoTimeout.

**Q. How do you shut a server down gracefully?**
Close the listener (Accept then returns an error), stop accepting, and wait for active handlers with a sync.WaitGroup. With context.Context you can signal handlers to finish.

**Q. TCP is a byte stream. What does that mean for your code?**
One Write on one side can arrive as several Reads on the other, or several Writes as one Read. You need framing: newline-delimited lines (what bufio.Scanner does here) or a length prefix.

**Q. Stateless vs stateful: which scales better and why?**
Stateless, because any server instance can handle any request, so you can add instances behind a load balancer. State is moved to a database or cache such as Redis.

# Part D: Likely exam questions for Unit 1

These are typical of how Unit 1 topics are asked. Check them against past CACS355 papers and note any question that repeats. The pointer tells you where the answer is in this guide.

## Short questions

| # | Question | Answer pointer |
|---|---|---|
| 1 | What is network programming? List its features. | A1 definition + A2 features (write 6-8 points) |
| 2 | Explain the scope of network programming. | A2 scope table, with examples |
| 3 | Differentiate between TCP and UDP. | A1 TCP vs UDP table (6+ rows) |
| 4 | What is a socket? What is a port? Differentiate them. | A1 key terms |
| 5 | Why is Java suitable for network programming? | B1 list |
| 6 | List the tools used in network programming and explain any three. | A3 tools table |
| 7 | Differentiate between a client and a server application. | A4 table |
| 8 | Differentiate iterative and concurrent servers. | A5 issue 2 |
| 9 | Differentiate stateful and stateless servers with examples. | A5 issue 4 |
| 10 | Differentiate client-server and peer-to-peer models. | A5 table |
| 11 | What are the issues of server complexity and privilege? | A5 issue 5 (5 points) |
| 12 | Explain 2-tier and 3-tier architecture. | A5 tiered architectures + simple diagram |

## Long questions

| # | Question | Answer pointer |
|---|---|---|
| 1 | Explain the client-server model with a diagram. List its advantages and disadvantages. | A5: definition, diagram, characteristics, advantages/disadvantages table |
| 2 | Explain the four types of servers based on connection and concurrency. | A5 issues 1-3, the 2x2 table, one example each |
| 3 | Explain the steps of TCP client-server communication with a diagram. Write a simple program. | A4 steps + diagram, then Program 1 (B3) |
| 4 | Write a client-server program in Java where the client sends a message and the server replies. | Program 1 (B3) |
| 5 | Write a multithreaded server program in Java that can handle multiple clients. | Program 2 (B4), mention thread pool |
| 6 | Write a UDP client and server program in Java. How is it different from TCP? | Program 3 (B5) + TCP vs UDP table |
| 7 | Discuss the software design issues of client-server applications. | A5 issues 1-7, short paragraph each |

> **Answer-writing pattern for theory questions**
>
> 1. One-line definition. 2. A diagram (even a simple box-and-arrow one). 3. Points or a comparison table. 4. A real example (HTTP, DNS, FTP...). 5. A one-line conclusion. Tables and diagrams score well and are quick to write.

# Part E: Java vs Go cheat sheet (Unit 1)

| Task | Java | Go |
|---|---|---|
| Start a TCP server | ServerSocket s = new ServerSocket(5000); | ln, err := net.Listen("tcp", ":5000") |
| Accept a client | Socket c = s.accept(); | conn, err := ln.Accept() |
| Connect as a client | Socket c = new Socket("localhost", 5000); | conn, err := net.Dial("tcp", "localhost:5000") |
| Read a line | new BufferedReader(new InputStreamReader(c.getInputStream())).readLine() | bufio.NewScanner(conn); sc.Scan(); sc.Text() |
| Write a line | new PrintWriter(c.getOutputStream(), true).println(s) | fmt.Fprintln(conn, s) |
| Other side closed | readLine() returns null | Scan() false / err == io.EOF |
| Serve concurrently | new Thread(new Handler(c)).start(); | go handle(conn) |
| Limit concurrency | Executors.newFixedThreadPool(n) | buffered channel semaphore |
| UDP server socket | new DatagramSocket(5002) | net.ListenPacket("udp", ":5002") |
| UDP receive | socket.receive(packet) | n, addr, err := conn.ReadFrom(buf) |
| UDP reply | socket.send(new DatagramPacket(b, b.length, addr, port)) | conn.WriteTo(b, addr) |
| Timeout | socket.setSoTimeout(ms) | conn.SetReadDeadline(time.Now().Add(d)) |
| Close | c.close() / try-with-resources | defer conn.Close() |
| Error handling | try { } catch (IOException e) { } | if err != nil { } |

**Next:** Unit 2 (InetAddress, Inet4Address/Inet6Address, NetworkInterface, SpamCheck, log file processing) with the same Java-then-Go layout.